torch.jit.trace_module#

https://pytorch.org/docs/stable/generated/torch.jit.trace_module.html#torch.

Description:

Trace a module and return an executable ScriptModule that will be optimized using just-in-time compilation. When a module is passed to torch.jit.trace, only the forward method is run and traced. With trace_module, you can specify a dictionary of method names to example inputs to trace (see the inputs) argument below. See torch.jit.trace for more information on tracing. mod (torch.nn.Module) – A torch.nn.Module containing methods whose names are specified in inputs. The given methods will be compiled as a part of a single ScriptModule. inputs (dict) – A dict containing sample inputs indexed by method names in mod. The inputs will be passed to methods whose names correspond to inputs' keys while tracing. { 'forward' : example_forward_input, 'method2': example_method2_input}

Function Signature

```
torch.jit.trace_module(mod, inputs, optimize=None,
check_trace=True, check_inputs=None, check_tolerance=1e-05,
strict=True, _force_outplace=False, _module_class=None,
_compilation_unit=<torch.jit.CompilationUnit object>,
example_inputs_is_kwarg=False, _store_inputs=True)[source]#

Parameters

Keyword Arguments

Returns
```

Parameters

Keyword

check_trace (bool, optional) – Check if the same inputs run through traced code produce the same outputs. Default: True. You might want to disable this if, for example, your network contains non- deterministic ops or if you are sure that the network is correct despite a checker failure. check_inputs (list of dicts, optional) – A list of dicts of input arguments that should be used to check the trace against what is expected. Each tuple is equivalent to a set of input arguments that would be specified in inputs. For best results, pass in a set of checking inputs representative of the space of shapes and types of inputs you expect the network to see. If not specified, the original inputs are used for checking check_tolerance (float, optional) – Floating-point comparison tolerance to use in t

Returns:

A ScriptModule object with a single forward method containing the traced code. When func is a torch.nn.Module, the returned ScriptModule will have the same set of sub-modules and parameters as func.

Code Examples

```
import torch
import torch.nn as nn
```

```
class Net(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv = nn.Conv2d(1, 1, 3)

    def forward(self, x):
        return self.conv(x)

    def weighted_kernel_sum(self, weight):
        return weight * self.conv.weight


n = Net()
example_weight = torch.rand(1, 1, 3, 3)
example_forward_input = torch.rand(1, 1, 3, 3)

# Trace a specific method and construct `ScriptModule` with
# a single `forward` method
module = torch.jit.trace(n.forward, example_forward_input)

# Trace a module (implicitly traces `forward`) and construct a
# `ScriptModule` with a single `forward` method
module = torch.jit.trace(n, example_forward_input)

# Trace specific methods on a module (specified in `inputs`),
constructs
# a `ScriptModule` with `forward` and `weighted_kernel_sum`
methods
inputs = {
    "forward": example_forward_input,
    "weighted_kernel_sum": example_weight,
}
module = torch.jit.trace_module(n, inputs)
```

Detailed Documentation

Documentation

Trace a module and return an executable ScriptModule that will be optimized using just-in-time compilation.

When a module is passed to torch.jit.trace, only the forward method is run and traced. With trace_module, you can specify a dictionary of method names to example inputs to trace (see the inputs) argument below.

See torch.jit.trace for more information on tracing.

mod (torch.nn.Module) – A torch.nn.Module containing methods whose names are specified in inputs. The given methods will be compiled as a part of a single ScriptModule.

inputs (dict) – A dict containing sample inputs indexed by method names in mod. The inputs will be passed to methods whose names correspond to inputs' keys while tracing. { 'forward' : example_forward_input, 'method2': example_method2_input}

check_trace (bool, optional) – Check if the same inputs run through traced code produce the same outputs. Default: True. You might want to disable this if, for example, your network contains non- deterministic ops or if you are sure that the network is correct despite a checker failure.

check_inputs (list of dicts, optional) – A list of dicts of input arguments that should be used to check the trace against what is expected. Each tuple is equivalent to a set of input arguments that would be specified in inputs. For best results, pass in a set of checking inputs representative of the space of shapes and types of inputs you expect

the network to see. If not specified, the original inputs are used for checking

check_tolerance (float, optional) – Floating-point comparison tolerance to use in the checker procedure. This can be used to relax the checker strictness in the event that results diverge numerically for a known reason, such as operator fusion.

example_inputs_is_kwarg (bool, optional) – This parameter indicate whether the example inputs is a pack pack of keyword arguments. Default: False.

A ScriptModule object with a single forward method containing the traced code. When func is a torch.nn.Module, the returned ScriptModule will have the same set of sub-modules and parameters as func.

Example (tracing a module with multiple methods):